Planned architecture

UML diagrams of the iOS app, the back end and the database, and the hosting options compared for F-11 (option C chosen). Drawn from the decisions (ADRs) and stories in `docs/`.

Version 1.3 · 3 November 2026 · own API, Postgres and S3 at Scaleway; signatures with Documenso · kept up to date by US-004, US-006, US-050 and US-070

At a glance

Front end	One iOS app in Expo · React Native · TypeScript, built in the cloud with EAS (no Mac needed). Installed on the two staff iPhones by Ad Hoc distribution. Online only: no business data is kept on the phones.
Back end	The developer's own API (TypeScript, Fastify) at Scaleway, Paris: logins, a check on every request, 5-minute links to private files, PDF rendering. Staging on his homelab.
Database	Managed PostgreSQL, 17 tables. Status changes through SQL functions only; signed data insert-only; every write audited.
Signatures	Self-hosted Documenso: the client draws once, it fills every page, and the PDF is sealed with the company's certificate.
Contract PDF	Rendered by the API with Chromium, in the language chosen in the preview. Archived with its SHA-256.
Operations	Nightly encrypted backup to a second EU provider, Sentry EU without personal data, deployments by CI.
Accounts	Apple: the company. Expo, GitHub, Scaleway, backups, Sentry: the developer (F-02), who pays the hosting (F-07).
Still open	F-14 Q3, F-15, F-18; the Documenso spike (US-009); legal checks L-01 to L-07. F-11 decided: C (pages 14–15).

Contents

System overview: what runs where 2
Build, release and development 3
Front end: the iOS app and the web app 4
Back end: the developer's API and Documenso 5
Database: the 17 tables 6
Database detail: fleet, photos, damage, maintenance 7
Database detail: clients, contracts and their terms 8
Database detail: what each handover records 9
Lifecycle of a contract 10
Lifecycle of a vehicle 11
Départ: from choosing the car to the sealed PDF 12
Retour: check-in, mandatory signature, final PDF 13
Why an API between the phone and Postgres (F-11) 14
Hosting options compared (F-11: C chosen) 15

How to read the diagrams

Deployment diagram p. 2–3
where each part runs, and who calls whom

Component diagram p. 4–5
the parts of the software, and what uses what

Class diagram p. 6–9
the tables, their fields and their links

State machine p. 10–11
the statuses, and what moves a record from one to the next

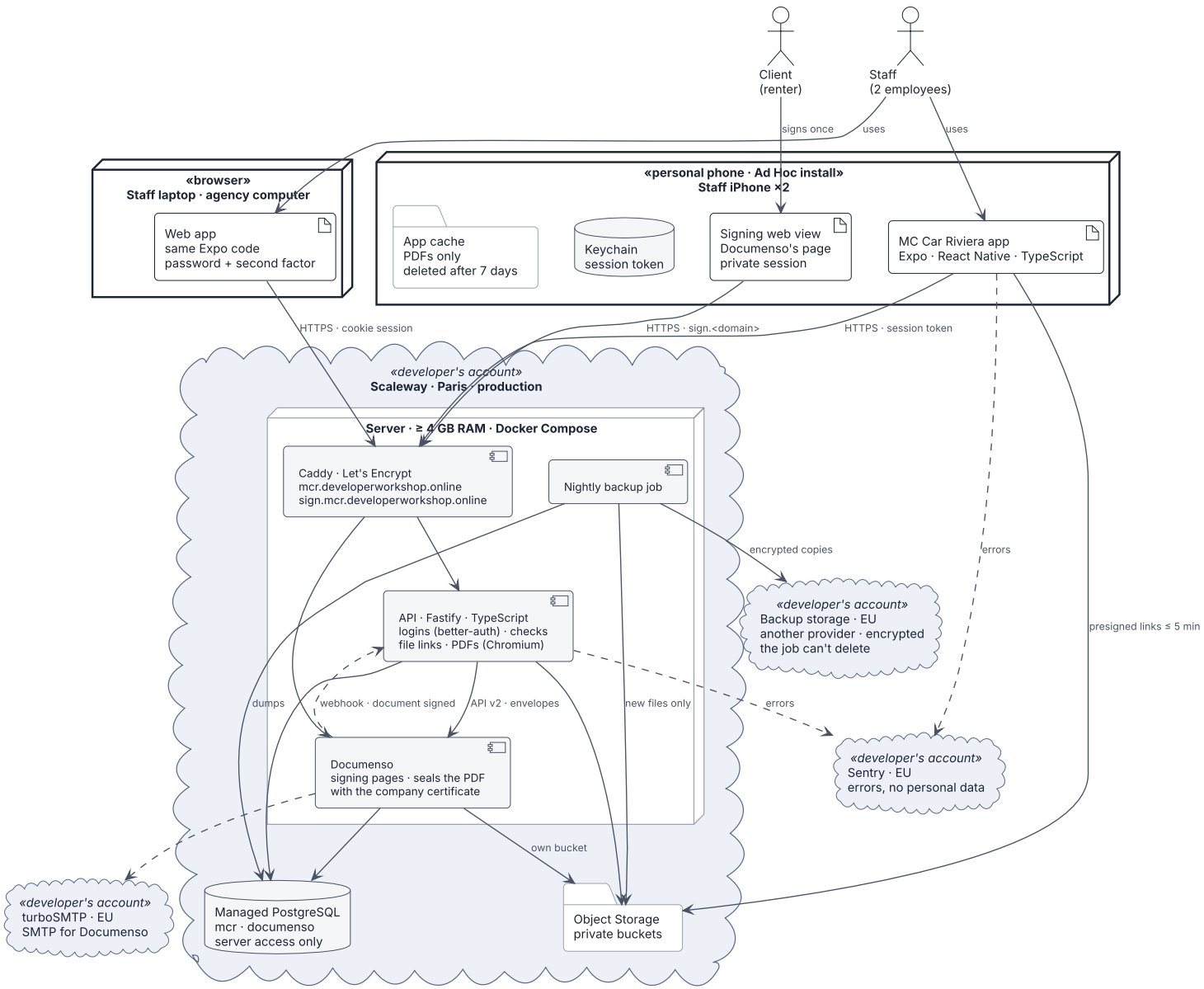
Sequence diagram p. 12–13
one workflow, message by message, from top to bottom

Colours

- Company's account (Apple)
- Developer's accounts (F-02)
- Phone or computer

2 System overview: what runs where

UML deployment diagram · the app in daily use



Company's account (Apple)

Developer's accounts (F-02)

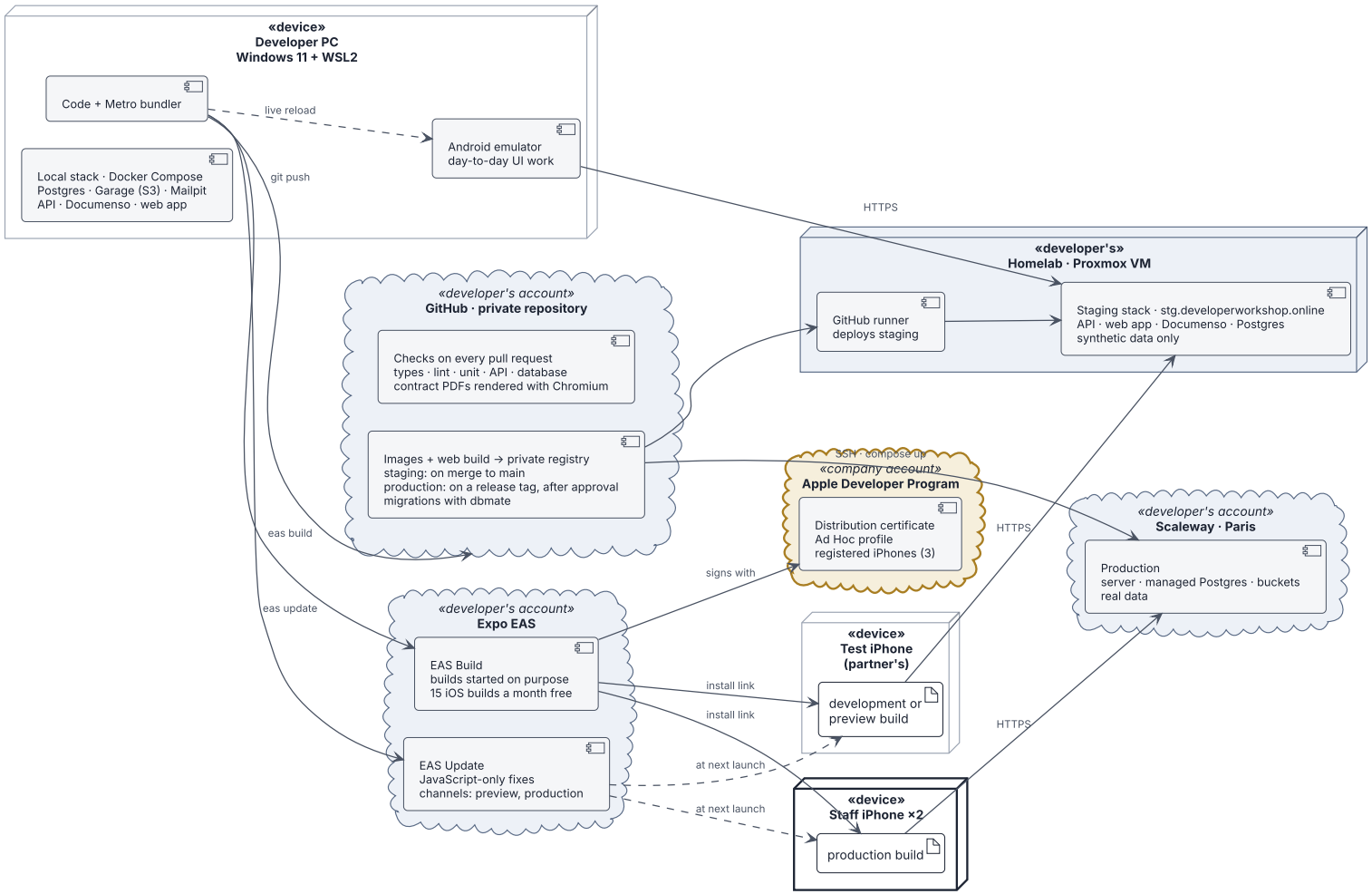
Phone or computer

Key points

- The same app runs on the iPhones and, as a web app, on the staff laptops and the agency computer (F-19). No local database and no offline mode; only PDFs to share sit in the iPhone's cache, for 7 days at most.
- Every call goes over HTTPS to the developer's own API at Scaleway, in Paris, at mcr.developerworkshop.online. The app holds only its session (Keychain on the iPhone, a protected cookie on the web), never a database password, a storage key or Documenso's token.
- The client signs on Documenso's page, opened in a private web view. Documenso seals the PDF and tells the API, which stores it and closes the step in one transaction.
- Files are private. The app reaches them through presigned links, issued by the API, that expire within 5 minutes.
- A nightly job on the server copies both databases and the new files, encrypted, to a second EU provider. Its key can't delete anything there.

3 Build, release and development

UML deployment diagram · from the developer's PC to the staff phones



Company's account (Apple)

Developer's accounts (F-02)

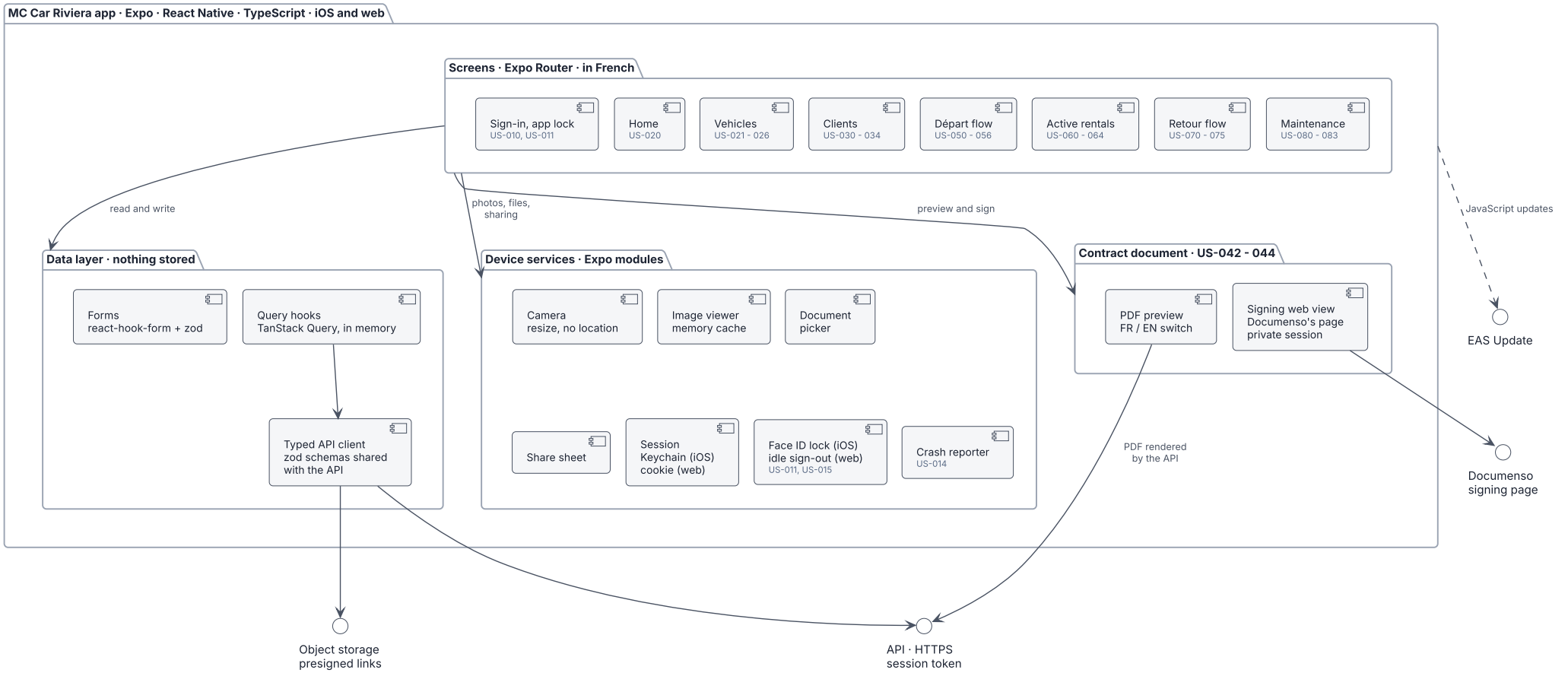
Phone or computer

Key points

- No Mac: EAS Build compiles the iOS app in the cloud. Builds are started on purpose; the free plan includes 15 iOS builds a month.
- Every build is installed by Ad Hoc internal distribution, signed with the company's Apple certificate. EAS holds it through the developer's Admin access; if the collaboration ends, the company revokes it and issues a new one.
- Staging runs on a VM on the developer's homelab, with synthetic data only. It serves the test iPhone and the Android emulator; production, at Scaleway, serves only the two staff phones.
- CI checks every pull request, including contract PDFs rendered with Chromium. Staging is deployed on merge to main; production on a release tag after manual approval. JavaScript-only fixes go out as EAS Updates.

4 Front end: the iOS app and the web app

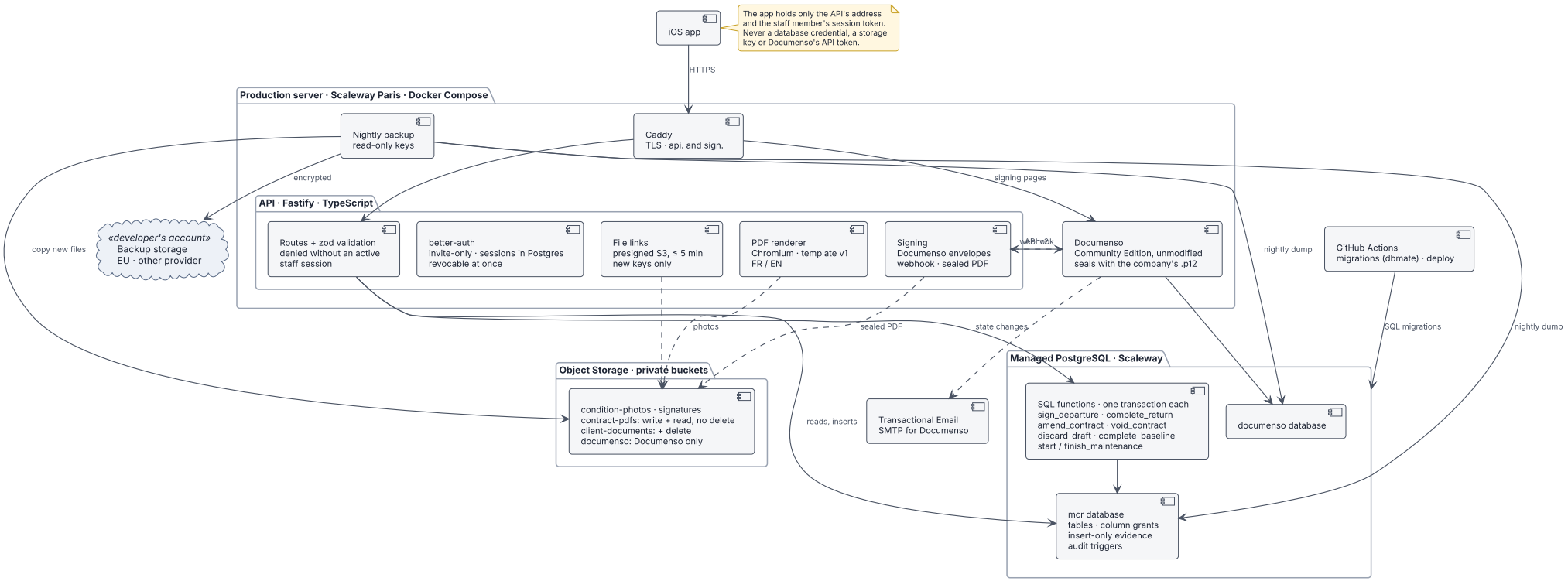
UML component diagram · one Expo codebase for iOS and the web; each screen lists the stories that build it



- One codebase for the iPhone and the web (F-19). Screens are in French, built with translation keys so English can be added later.
- Server data is kept in memory only (TanStack Query, no persister). Forms are checked with zod before anything is sent.
- The app renders no PDF and has no signature pad: it shows the API's preview, with a French / English switch, then opens Documenso's signing page in a private web view.
- Photos are resized and stripped of their location before upload. The session token is in the Keychain. Crash reports carry no personal data.

5 Back end: the developer's API and Documenso

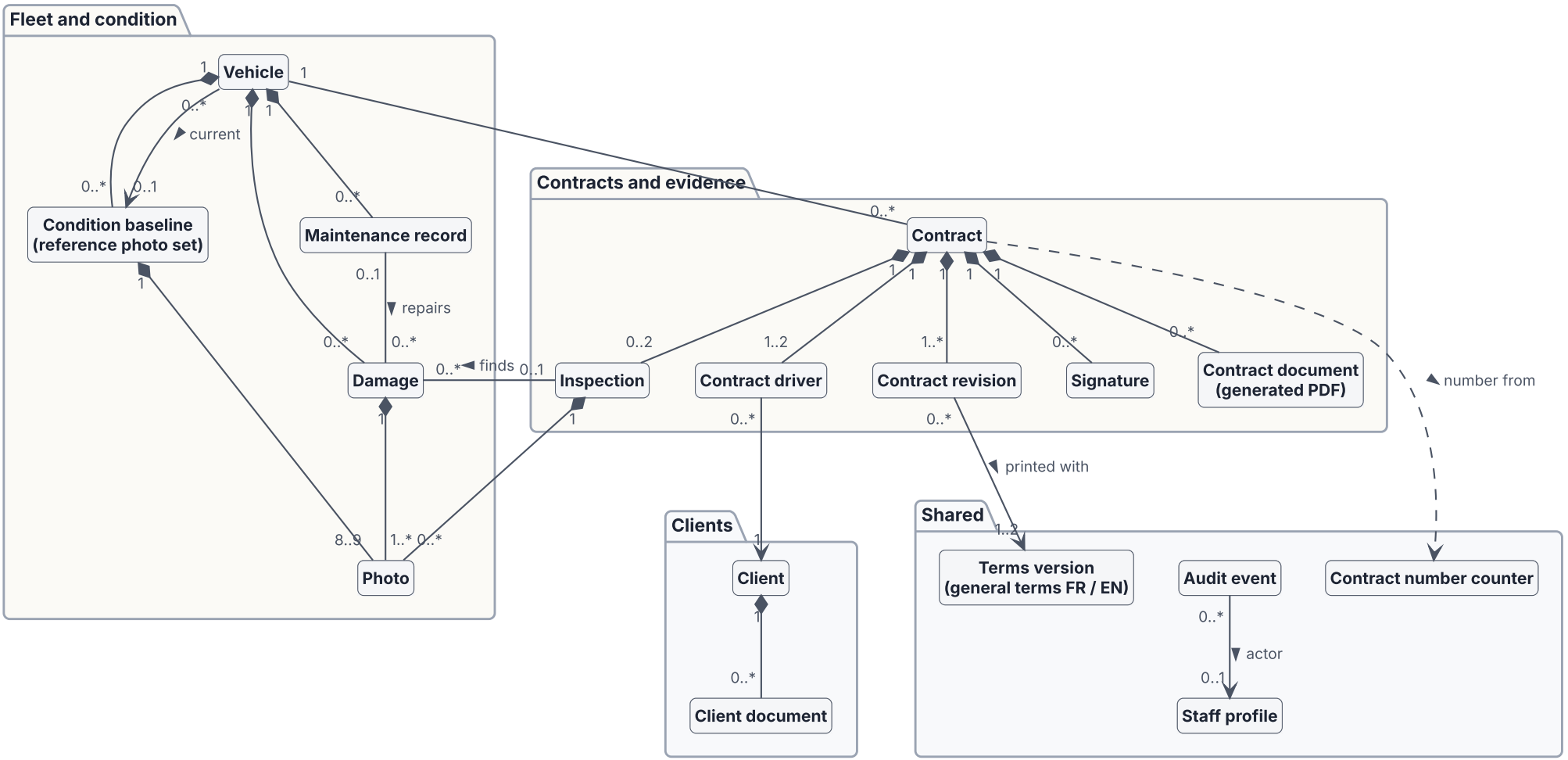
UML component diagram · Scaleway, Paris; the API checks every request, the database enforces the rules



- Every route needs an active staff session (better-auth), denied by default; a test lists every route. Revoking a session works at once.
- Every status change (sign, close, amend, void, maintenance) is one SQL function running in one transaction, so two phones can't rent the same car. Status columns are read-only for the API's role, evidence tables accept inserts only, and audit triggers record every write.
- Documenso runs unmodified next to the API. The API creates the envelopes, receives the webhook, and stores the sealed PDF before the contract changes state.
- The API's storage key can't delete evidence. Database credentials, storage keys, Documenso's token and the signing certificate live only on the server and in protected CI environments.

6 Database: the 17 tables

UML class diagram · overview; the fields are on the next three pages

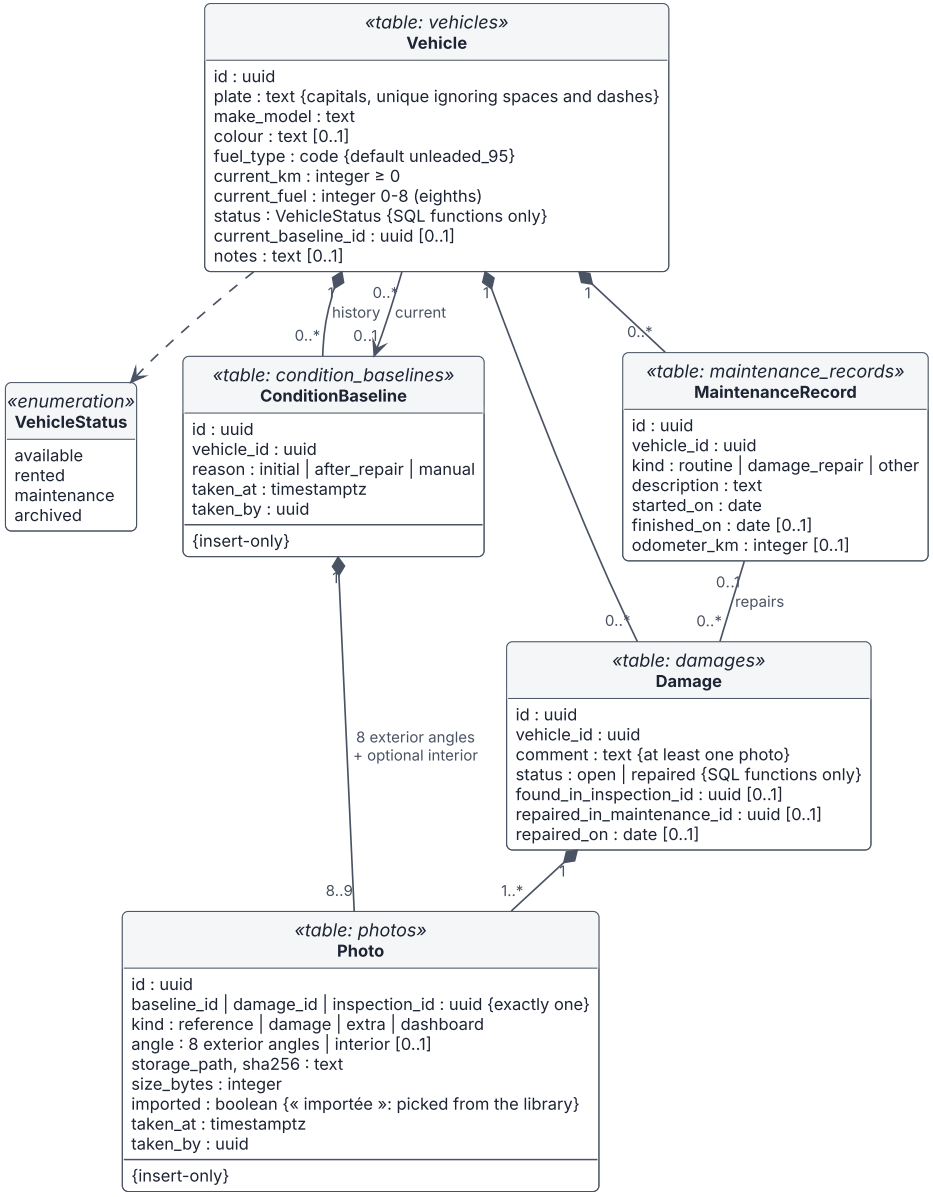


- Working table names from US-006. Once the migrations exist they are the reference, and these diagrams follow them.
- A contract is a header plus numbered revisions. Revision 1 is the Départ as signed; each amendment adds a revision with a full snapshot.
- Condition evidence is reused: a Départ's inspection points to the vehicle's current reference photo set and to the damage open at that moment (drawn on « What each handover records »).

Notation ◆— part of (belongs to one owner) —› refers to **1 · 0..1 · 0..*** how many **[0..1]** optional field **{...}** rule

7 Database detail: fleet, photos, damage, maintenance

UML class diagram · vehicles, condition_baselines, photos, damages, maintenance_records



Key points

- A reference set has the 8 exterior angles, all required, plus an optional interior photo. It is taken with the in-app camera.
- The optional dashboard photo (km and fuel) belongs to a handover's inspection, not to the reference set: reused later, it would show an old odometer reading.
- Finishing a damage repair closes the repaired damage. The car can be rented again only once a new complete reference set exists.
- Photos and reference sets are insert-only. Statuses change only through SQL functions.

Notation

◆— part of (belongs to one owner)

—› refers to

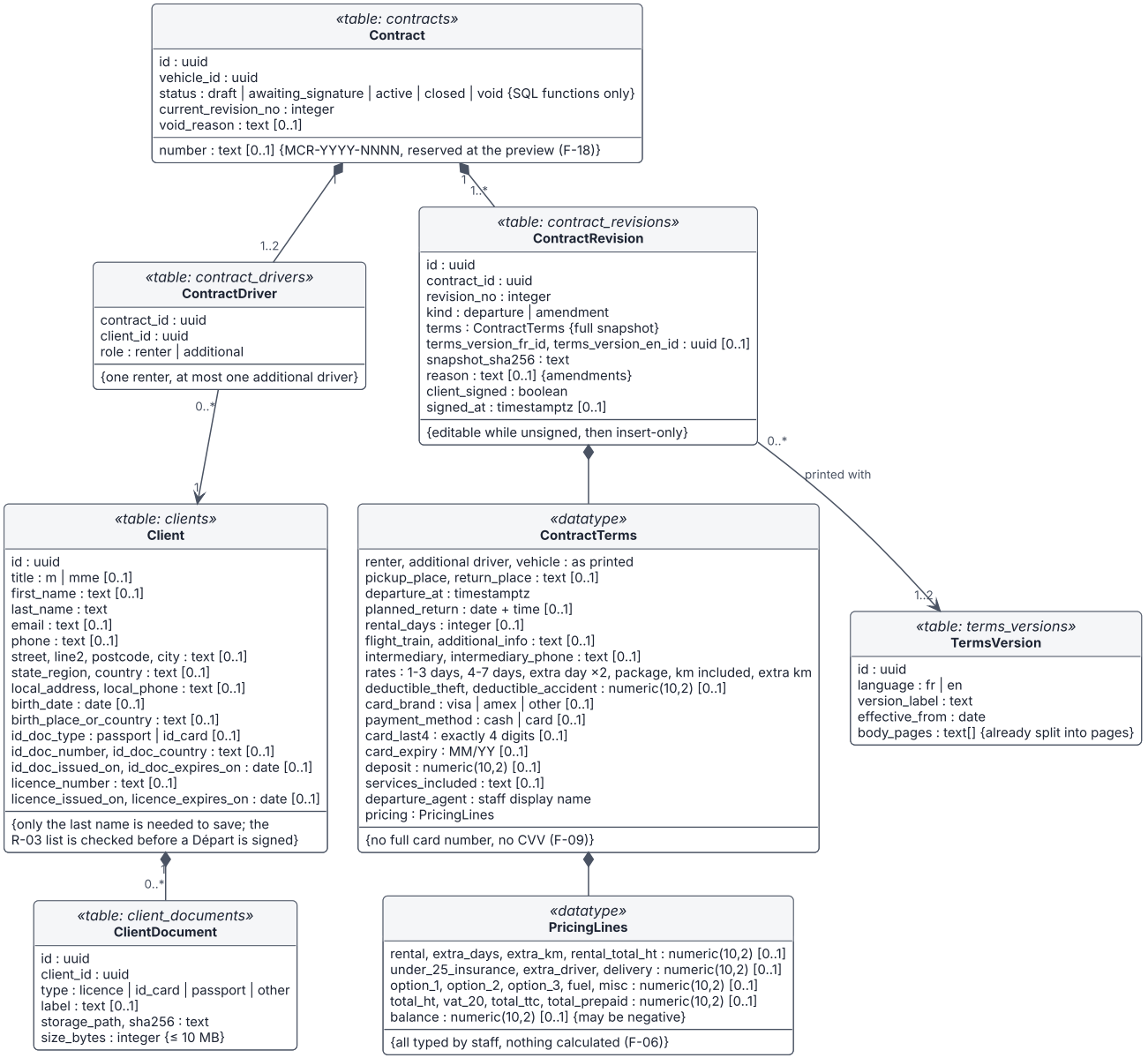
1 · 0..1 · 0..* how many

[0..1] optional field

{...} rule

8 Database detail: clients, contracts and their terms

UML class diagram · clients, client_documents, contracts, contract_drivers, contract_revisions, terms_versions



Key points

- Signing freezes revision 1: the client and vehicle values are copied as printed, so later edits to a client or a vehicle never change a signed contract.
- Card: brand, last four digits and expiry only. No column can hold a full card number or a CVV (F-09, PCI DSS).
- Every amount is typed by staff (F-06). The app computes only km driven and extra km.
- Each revision records the French and English terms versions in force when it was signed.

Notation

◆— part of (belongs to one owner)

—› refers to

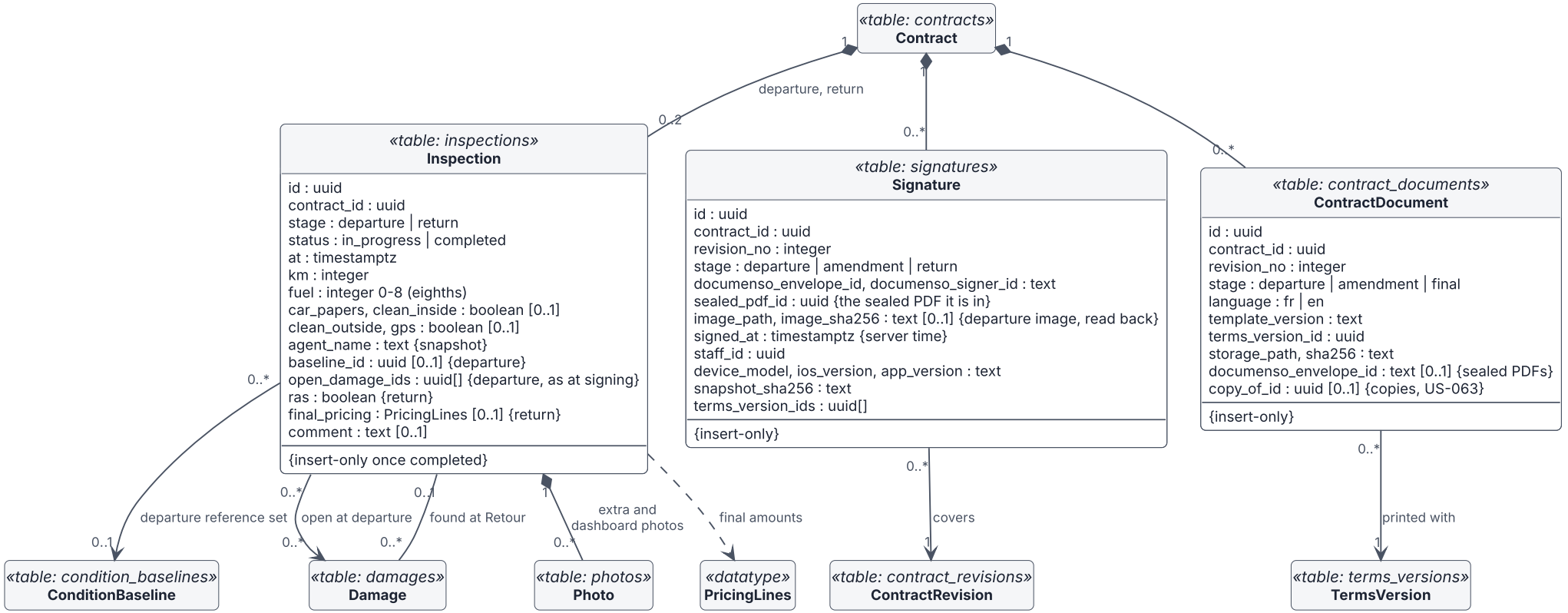
1 · 0..1 · 0..* how many

[0..1] optional field

{...} rule

9 Database detail: what each handover records

UML class diagram · inspections, signatures, contract_documents (grey: drawn in full on earlier pages)

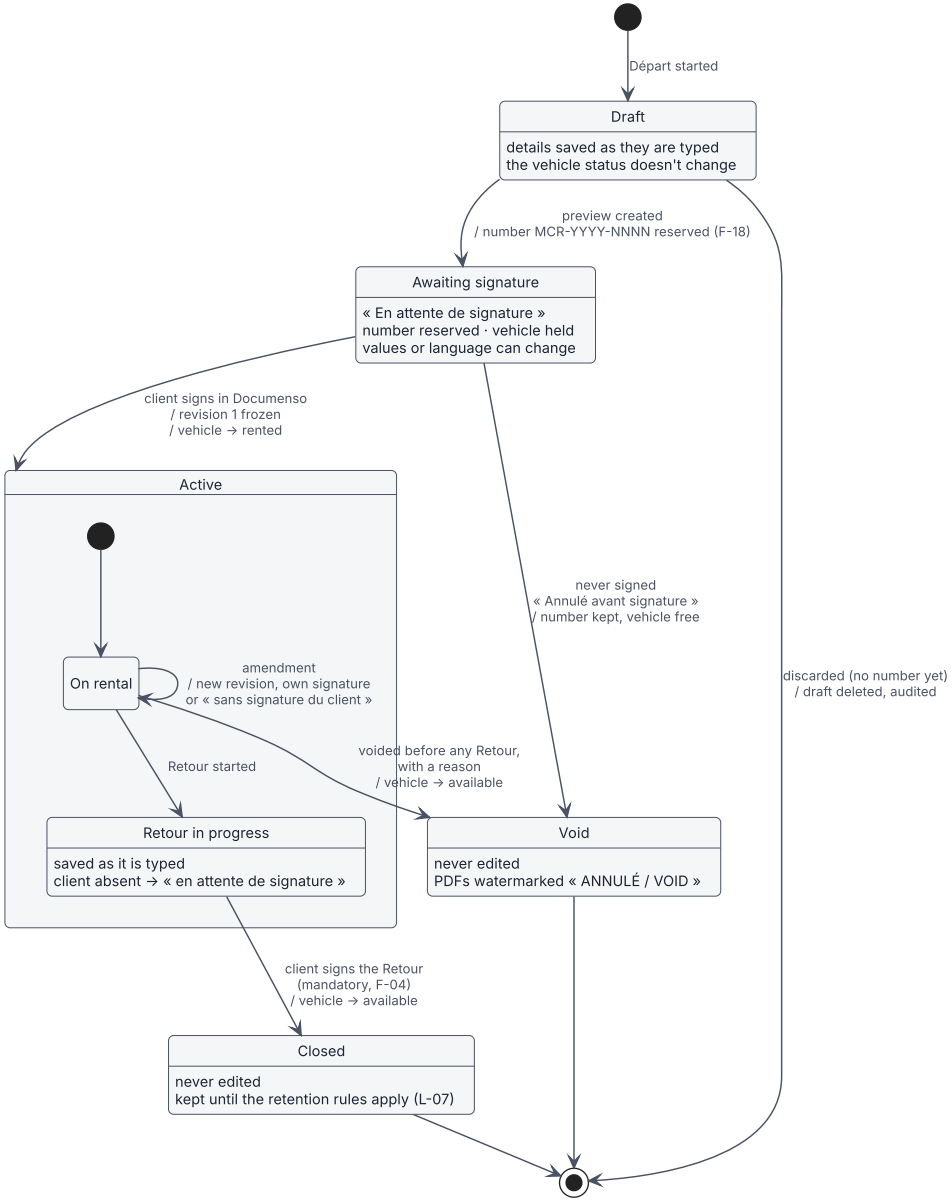


- Each signature has an evidence record: Documenso's envelope and signer IDs, the sealed PDF it is in, server time, staff member, device, iOS and app versions, and the hash of the data that was signed. The departure image is read back for the final PDF (F-17).
- Every PDF is archived with its SHA-256, its template version and its terms version. Copies (US-063) point to the sealed original.
- Completed inspections, signatures and PDF records are insert-only. Until the client signs, the Retour's km, fuel, amounts and comments can still be corrected; whether a photo or a damage entry can be taken back is open (R-10 Q5).

Notation ◆— part of (belongs to one owner) —› refers to **1 · 0..1 · 0..*** how many **[0..1]** optional field **{...}** rule

10 Lifecycle of a contract

UML state machine · every transition is an SQL function called by the API, recorded in the audit trail

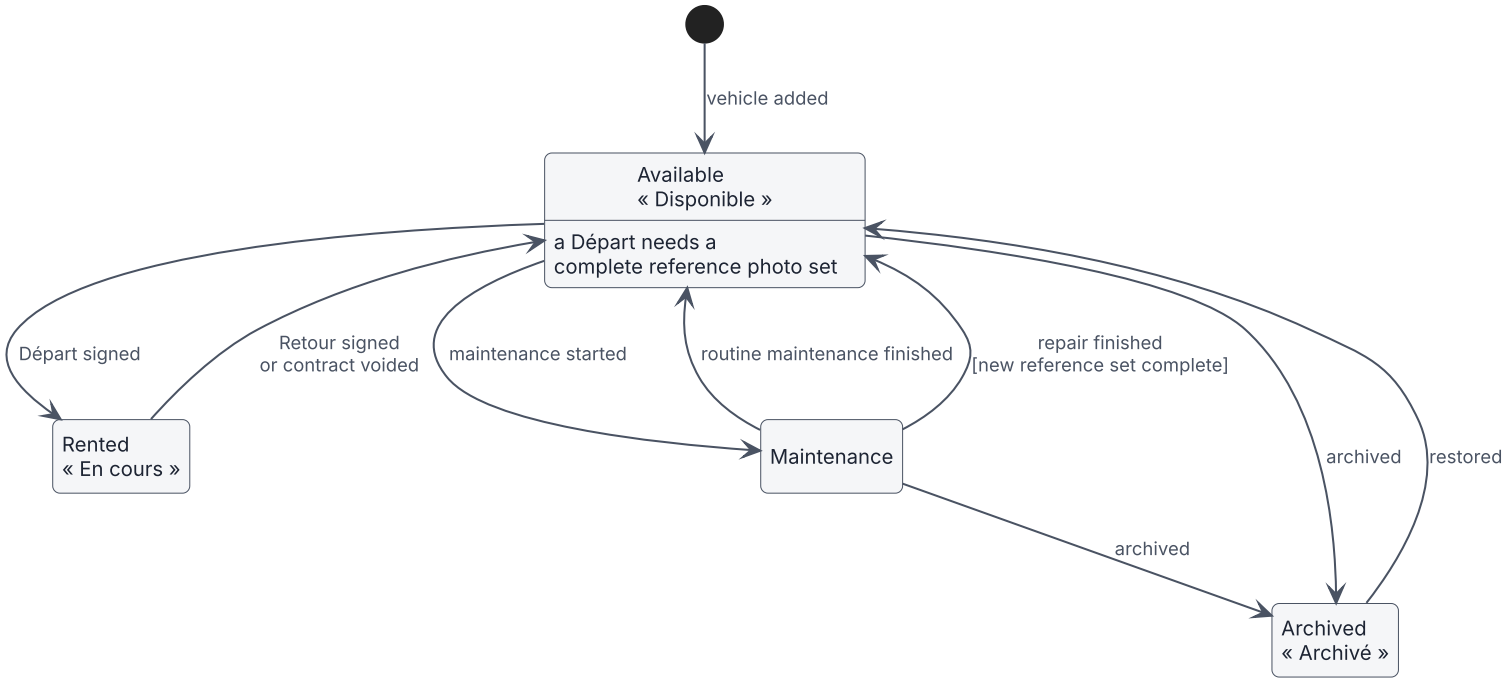


Key points

- A draft never changes the vehicle's status. Before the preview it can be discarded; once the preview reserves a number, it can only be cancelled, and is kept as « Annulé avant signature » (F-18).
- Signing the Départ confirms the number, freezes revision 1 and marks the car rented, in one transaction.
- An amendment adds a revision, signed by the client or recorded « sans signature du client » (default for R-11).
- A contract can be voided, with a reason, only before its Retour. The app never deletes a signed contract.

11 Lifecycle of a vehicle

UML state machine · the labels in « » are what staff see in the app

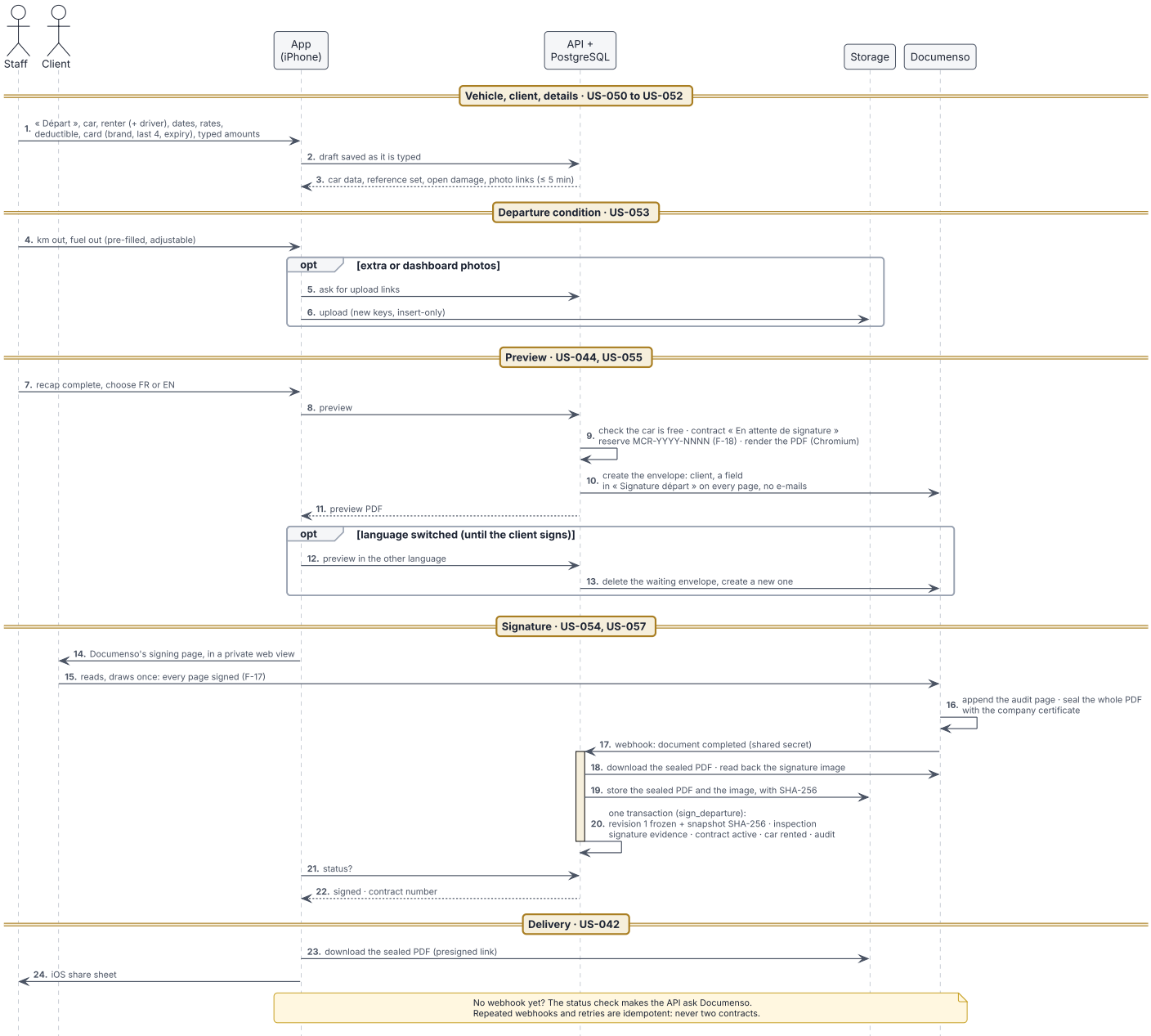


Key points

- A Départ is offered only for available cars with a complete reference photo set.
- Routine maintenance needs no new photos. After a damage repair, a new complete reference set is required before the car is available again.
- Only an available car, or one in maintenance, can be archived. « Restaurer » brings it back; nothing is deleted.
- Statuses change only through SQL functions, in the same transaction as the contract or maintenance change.

12 Départ: from choosing the car to the sealed PDF

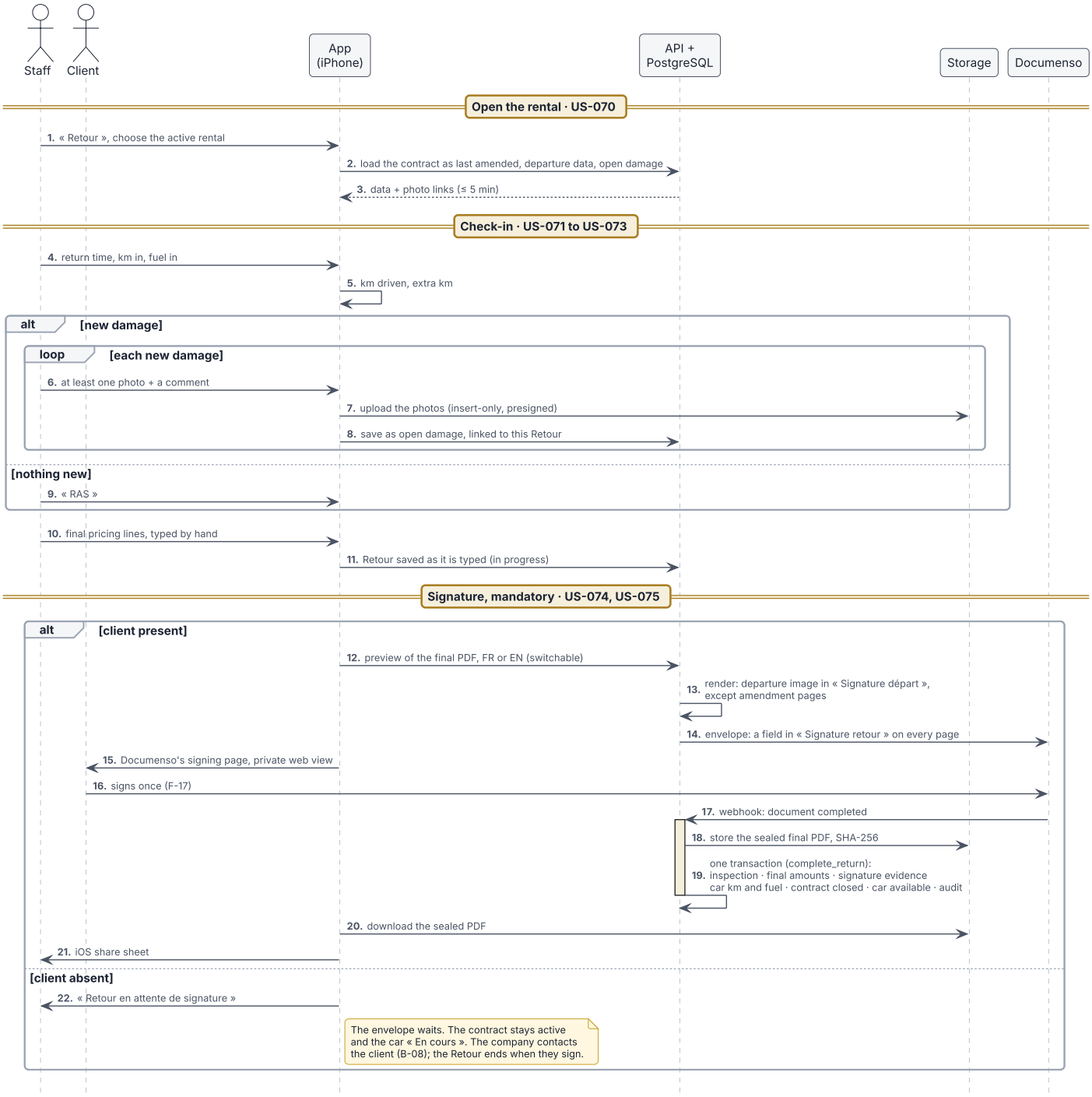
UML sequence diagram · US-050 to US-057, US-044



- The language is chosen in the preview, and can be switched until the client signs (F-17). The PDF carries its contract number already, so the number is reserved at the preview (F-18).
- The client draws once in Documenso, and the drawing fills the signature box of every page. Documenso seals the PDF with the company's certificate.
- The contract becomes active only once the sealed PDF is stored, in one transaction. If the car was taken meanwhile, nothing is signed in the app.

13 Retour: check-in, mandatory signature, final PDF

UML sequence diagram · US-070 to US-075, US-044



- The return signature is mandatory (F-04). Without the client, the Retour waits « en attente de signature » and the car stays « En cours ».
- Km driven and extra km are the only values the app computes. The final amounts are typed (F-06).
- In the final PDF the client signs the « Signature retour » box of every page in Documenso. The « Signature départ » boxes show the departure image read back from Documenso, except on amendment pages (F-08, F-17).

14 Why an API between the phone and Postgres (F-11)

Your question of 9 October, answered on 3 November with option C · the four options side by side

Traffic isn't what costs money

Two staff members and a few dozen requests per handover are nothing for any of these options. What costs money is storing the files, and the security layer someone has to build and run.

The phone can't talk to Postgres directly

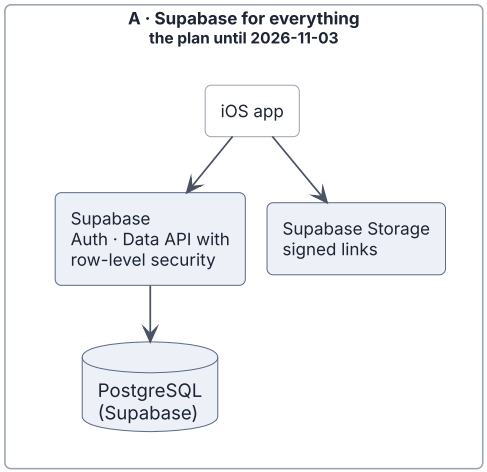
It would need a database password on two personal phones, and Postgres open to the internet. Something must sit in between: sign staff in, check every request against the access rules, and hand out short-lived links to private files. With option C, that layer is the developer's own API.

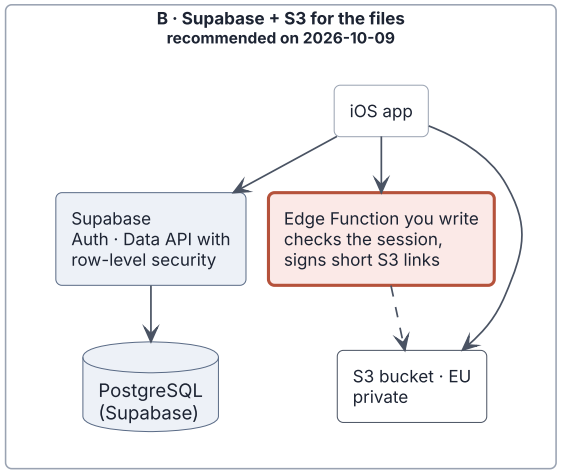
What C costs

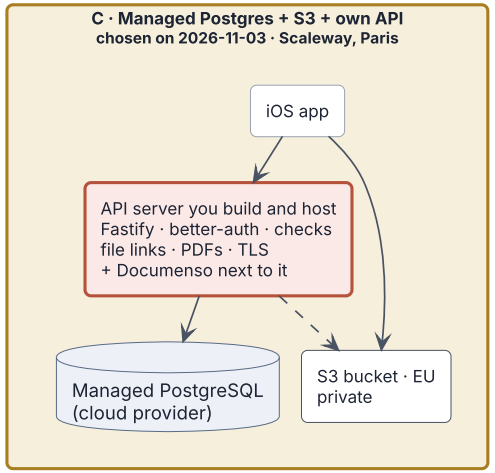
About €25–30 a month: the managed Postgres (about €11), a server with at least 4 GB of RAM for the API and Documenso (about €15), cents for the files. And weeks of extra work: logins, permissions, file links, deployment, monitoring.

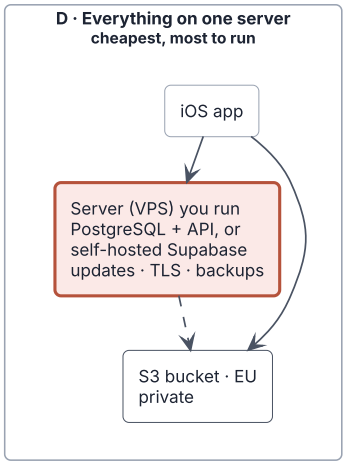
What C gives

No free-plan pause or quota, the data on infrastructure the developer controls, in France, one language from the app to the API, and room for Documenso next to the API.









Run by a provider · Built or run by the developer · S3-compatible bucket

15 Hosting options compared (F-11: C chosen)

Approximate list prices before VAT, checked on 9 October 2026 · every option stays portable (Postgres dump, S3-compatible files)

	A · Supabase for everything the plan until 3 November	**B · Supabase + S3 for the files** recommended on 9 October	**C · Managed Postgres + S3 + own API** chosen on 3 November	**D · Everything on one server** cheapest, most to run
What runs where	Logins, database, API and files on Supabase (EU)	Logins, database and API on Supabase Free; files in an EU S3-compatible bucket (e.g. Scaleway, Paris)	Managed Postgres, a small server for your own API, and an S3 bucket	One small server (e.g. Hetzner CX23, €5.49) with Postgres and an API, or self-hosted Supabase; files in S3
Per month, under 1 GB of files	€0	€0 + cents	≈ €17; ≈ €25–30 with Documenso on a 4 GB server	≈ €6–8
Per month after ≈ 200 rentals	$25 (Pro)	Cents: the text data stays under 500 MB for years	The same, plus cents for files	≈ €6–8
Extra work for you	None	1–2 days: one function for file links, and the bucket set-up	Weeks: logins, sessions, permissions, the API itself, TLS, monitoring	Days to set up, then updates, security, backups and monitoring of a server holding ID documents
Who patches and watches it	Supabase	Supabase and the bucket provider	The provider for the database; you for the API server	You
Limits to know	Free plan: no service guarantee; pauses after a week without activity	Free-plan limits for the database, as in A	None of the free-plan limits	Your one server is a single point of failure

Decision (3 November): C

Your own API server in TypeScript with Fastify (F-12), a managed Postgres and S3 storage, at Scaleway in Paris (F-14), with staging on your homelab. Signatures with a self-hosted Documenso next to the API. Recorded in ADR-0019 and ADR-0020; Supabase (ADR-0004) is superseded.

What C adds since 9 October

- A server with at least 4 GB of RAM, since Documenso needs 2 GB on its own: about €25–30 a month in all.
- An e-mail provider, which Documenso needs even with its e-mails to clients switched off (F-15).
- PDF rendering on the server with Chromium (F-13), and the nightly backup run from the server.

Still to watch

- One server: while it is down, no handover can be signed. Uptime alerts and the paper fallback cover it.
- Scaleway becomes a sub-processor in your data processing agreement with the company (L-03).

Prices: Supabase pricing page (Free and Pro plans) · Scaleway price lists for Object Storage (€0.01606 per GB a month, Paris, multi-AZ) and Managed Databases (DB-DEV-S, €0.0156 an hour) · Hetzner's cloud price change of 15 June 2026 (CX23 from €3.99 to €5.49 a month). Check them again before committing.